

Software Quality - First assignment

Susana Wang
73913

sw.wang@campus.fct.unl.pt

Madalena Alves
75150

mfo.alves@campus.fct.unl.pt

ABSTRACT

This paper presents a reverse engineering analysis of two Java systems: JHotDraw 5.2 and Apache Ant. The former is provided as a benchmark, while the latter was selected as the subject system. The analysis is conducted using the tools inFusion [5] and MetricsTree [4], focusing on size, complexity, coupling, and design disharmonies. The goal is to identify structural strengths and weaknesses that may impact the maintainability and evolution of both systems

1. INTRODUCTION

This report presents a reverse engineering analysis of JHotDraw 5.2, used as a common benchmark, and Apache Ant, the group's selected subject system. Both are Java systems analyzed using MetricsTree [4] and Infusion [5], focusing on size, complexity, coupling, and design disharmonies, with the goal of identifying structural weaknesses relevant to future maintainability and evolution [1][2].

2. SYSTEMS DESCRIPTION

2.1 Apache Ant

Apache Ant is a Java-based build automation tool widely used for compiling, packaging, and deploying software projects. It is designed as a standalone application and follows a modular architecture based on tasks and build scripts defined in XML. The analyzed version contains approximately 1000 classes and over 5000 methods, representing a system of moderate size. The project was imported into IntelliJ as a Java project without requiring execution. Metrics were extracted using inFusion [5] and MetricsTree [4].

2.2 Jhotdraw

JHotDraw 5.2 is a Java framework for building graphical user interfaces, particularly drawing editors. It provides reusable components for managing graphical objects, user interactions, and application structure. The system is smaller than Apache Ant and serves as a benchmark for comparison in this analysis. It was imported from an existing Eclipse project into IntelliJ, with Java 8 configuration. Metrics were collected using inFusion [5] and MetricsTree [4].

3. ANALYSIS

3.1 Apache Ant

System Overview

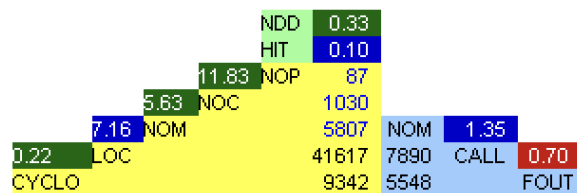


Figure 1

The Overview Pyramid of Apache Ant (Fig.1) provides a high-level characterization of the system in terms of size, complexity, and coupling.

From a size perspective, Apache Ant is a large-scale system, comprising approximately 1030 classes (NOC), 5807 methods (NOM), and 41617 lines of code (LOC) distributed across 87 packages (NOP). These values indicate a mature and feature-rich codebase, consistent with its role as a widely used build automation tool.

Regarding complexity, the system exhibits moderate logical complexity, as reflected by a Cyclomatic Complexity per LOC (CYCLO/LOC) of 0.22 [3]. This suggests that, on average, methods are not excessively complex and that control flow remains relatively manageable. Additionally, the LOC/NOM ratio (7.16) indicates that methods are generally short, which typically contributes positively to readability and maintainability[1].

In terms of structural organization, the NOM/NOC ratio (5.63) shows that classes contain a moderate number of methods, suggesting a balanced distribution of responsibilities across classes. There is no immediate indication of extreme class bloat at the system level.

Concerning inheritance, the metrics AHH (0.10) and ANDC (0.33) reveal shallow inheritance hierarchies with limited subclassing. This indicates that Apache Ant does not heavily rely on deep inheritance structures, which may simplify understanding but can also limit reuse through polymorphism [1].

Finally, coupling analysis highlights some noteworthy aspects. The CALL/NOM ratio (1.35) suggests a reasonable level of interaction between methods. However, the FOUT/CALL ratio (0.70) indicates a relatively high coupling dispersion, meaning that method calls are distributed across many different classes. This can increase the system's sensitivity to changes and may negatively impact maintainability.

Overall, Apache Ant can be characterized as a large but relatively well-structured system, with moderate complexity and shallow inheritance, though with non-negligible coupling dispersion that may pose challenges for evolution and maintenance.

3.1.1 Classic Metric View

Me...	Metrics...	Description	Value	Regula...
○ CLOC	Statistic	Lines of Code	1696	
○ CCC	G. Ann C...	Cognitive Complexity	178	[0..32]
○ CHVL	Halstead...	Halstead Volume	28624,...	
○ CHD	Halstead...	Halstead Difficulty	386,2459	
○ CHL	Halstead...	Halstead Length	3114	
○ CHEF	Halstead...	Halstead Effort	1105621...	
○ CHVC	Halstead...	Halstead Vocabulary	585	
○ CHER	Halstead...	Halstead Errors	16,5431	
○ WMC	Chidamb...	Weighted Methods Per Class	286	[0..12]
○ DIT	Chidamb...	Depth Of Inheritance Tree	2	[0..3]
○ CBO	Chidamb...	Coupling Between Objects	71	[0..14]
○ RFC	Chidamb...	Response For A Class	284	[0..45]
○ LCOM	Chidamb...	Lack Of Cohesion Of Methods	1	
○ NOC	Chidamb...	Number Of Children	0	[0..2]
○ NOA	Lorenz-K...	Number Of Attributes	49	[0..4]
○ NOO	Lorenz-K...	Number Of Operations	164	
○ NOOM	Lorenz-K...	Number Of Overridden Methods	1	[0..3]
○ NOAM	Lorenz-K...	Number Of Added Methods	122	
○ SIZE2	Li-Henry ...	Number Of Attributes And Methods	208	
○ NOM	Li-Henry ...	Number Of Methods	123	[0..7]
○ MPC	Li-Henry ...	Message Passing Coupling	11	
○ DAC	Li-Henry ...	Data Abstraction Coupling	9	
○ ATFD	Lanza-M...	Access To Foreign Data	0	[0..6]
○ NOPA	Lanza-M...	Number Of Public Attributes	0	[0..3]
○ NOAC	Lanza-M...	Number Of Accessor Methods	22	[0..4]
○ WOC	Lanza-M...	Weight Of A Class	0,5285	[0,5..1,0]
○ TCC	Bieinan-...	Tight Class Cohesion	0,0285	[0,33..1,0]
○ NCSS	Chr. Cle...	Non-Commenting Source Statements	777	[0..1000]
○ CMI	Maintain...	Maintainability Index	0,0	

Figure 2

The Classic Metric View (Fig.2) provides an overview of the system's structural properties, including size, complexity, coupling, cohesion, and inheritance. This figure is useful as a baseline, allowing the identification of metrics that fall outside recommended thresholds and may require further analysis.

The values indicate relatively high complexity levels, particularly in the Weighted Methods per Class (WMC = 286) and Response for Class (RFC = 284), both exceeding typical ranges. This suggests that some classes concentrate too many responsibilities, which increases the effort required to understand and maintain them. Coupling is also a concern. The Coupling Between Objects value (CBO = 71) is well above acceptable limits, indicating strong dependencies between classes. This reduces modularity and makes the system more sensitive to changes. Size-related metrics, such as Number of Methods (NOM = 123) and Number of Attributes (NOA = 49), further suggest the presence of large classes. These are often associated with design issues like God Classes or Large Class disharmonies [2]. Although cohesion metrics appear acceptable, they should be interpreted carefully, as high size and coupling values may hide underlying design problems.

Overall, this figure provides objective data to support the analysis and helps identify which metrics should be explored in more detail. It also serves as a starting point before moving to more focused views.

3.1.2 Build Metrics Distribution Chart

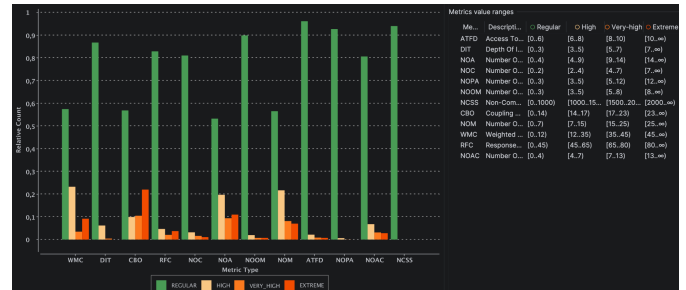


Figure 3

The Build Metrics Distribution Chart (Fig.3) shows how metric values are distributed across threshold categories (Regular, High, Very High, Extreme), providing a system-wide perspective. Most values fall within the "Regular" range, indicating that the majority of classes do not present significant issues. However, there are still noticeable portions in the "High" and "Very High" categories, particularly for metrics such as WMC (Fig. 6), CBO (Fig. 4), and NOM (Fig.4). The presence of even a small number of "Extreme" values is important, as these usually correspond to a few classes with disproportionately high complexity or size. These outliers can have a strong impact on the system's maintainability. This reflects a common pattern in software systems, where issues are concentrated in a small subset of components rather than evenly distributed [1]. This figure supports focusing the analysis on the most problematic elements. It also aligns with the goal of identifying components that stand out due to extreme values.

3.1.3 CBO Distribution Chart

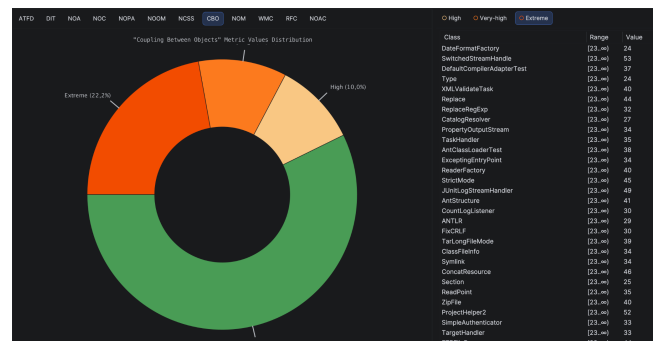


Figure 4

The CBO Distribution Chart (Fig.4) focuses specifically on coupling, one of the most critical factors influencing software maintainability. By isolating this metric, the chart provides a clearer understanding of how dependencies are distributed across the system.

The distribution indicates that a significant proportion of classes fall within the Regular range, suggesting that, for the majority of the system, coupling remains under control. This is a positive sign, as low coupling is generally associated with better modularity and easier maintenance. However, the presence of noticeable percentages in the High and Very High categories, along with some Extreme cases, raises important concerns.

Classes with high coupling values typically act as central hubs within the system [1], either depending on many other classes or

being heavily depended upon. This centralization introduces several risks. Changes to these classes are more likely to propagate throughout the system, increasing the likelihood of unintended side effects. Additionally, high coupling complicates testing and refactoring, as it becomes more difficult to isolate components and reason about their behavior independently.

From an architectural perspective, these highly coupled classes may indicate poor separation of concerns or insufficient modularization. Their identification is therefore critical, as they represent prime candidates for refactoring strategies such as decoupling, interface extraction, or decomposition into smaller components.

This figure isolates coupling as a standalone risk factor, rather than combining it with other metrics. It provides clear, focused evidence of architectural weaknesses and supports the identification of specific components that may compromise the system’s long-term maintainability.

3.1.4 NOM Distribution Chart

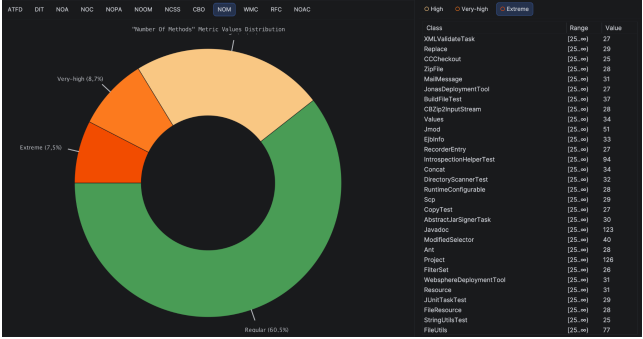


Figure 5

The NOM Distribution Chart (Fig.5) provides insight into how functionality is distributed across classes by analyzing the Number of Methods metric. This perspective is essential for understanding class size and identifying potential violations of design principles.

The chart shows that a large proportion of classes fall within the Regular range, indicating that most classes have a reasonable number of methods and are likely well-scoped. This suggests that, for the majority of the system, functionality is distributed in a balanced manner. However, the existence of classes in the High and Very High categories highlights the presence of components with an excessive number of methods.

Such classes are often problematic, as they tend to accumulate multiple responsibilities, thereby violating the Single Responsibility Principle [2]. As a result, they become harder to understand, test, and maintain. These classes may also act as aggregation points for unrelated functionality, further increasing their complexity and reducing cohesion.

When considered alongside other metrics, such as WMC, high NOM values can provide additional insights into the nature of complexity. For instance, a class with a high number of methods but moderate complexity may consist of many simple methods, whereas a class with both high NOM and high WMC likely combines size with significant logical complexity, making it particularly problematic.

This figure is therefore valuable for identifying size-related design issues at the class level. It helps detect candidates for Large Class

or God Class disharmonies and complements complexity metrics by providing a more nuanced understanding of how functionality is structured within the system.

3.1.5 WMC Distribution Chart

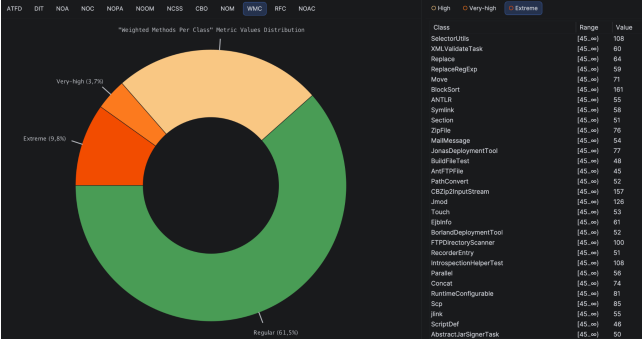


Figure 6

The WMC Distribution Chart (Fig.6) is one of the most critical visualizations for assessing class-level complexity, as it directly reflects the cumulative complexity of methods within each class. This metric is closely related to the effort required to understand, maintain, and modify the system.

The distribution indicates that most classes fall within the Regular category, suggesting that overall complexity is manageable for a large portion of the system. Nevertheless, the presence of classes in the High and Very High categories, along with some Extreme values, is particularly concerning. These classes represent complexity hotspots that can significantly impact the maintainability of the system.

High WMC values typically indicate classes with complex logic, excessive responsibilities, or both [3]. Such classes are often associated with design issues such as Brain Classes or God Classes [2], where a single component becomes overloaded with functionality. These elements are difficult to understand and are more prone to defects, as their complexity increases the likelihood of errors and reduces the effectiveness of testing.

The identification of Extreme WMC values is especially important, as these cases represent the most critical points in the system. According to reverse engineering best practices, these elements should be prioritized for detailed inspection, potentially at the method level, to understand the sources of complexity and evaluate the need for refactoring .

This figure is arguably one of the most important in the analysis, as it directly captures cognitive complexity. It provides a clear basis for prioritizing investigation efforts and aligns closely with the objective of identifying the most problematic components in the system.

3.2 Jhotdraw

System Overview

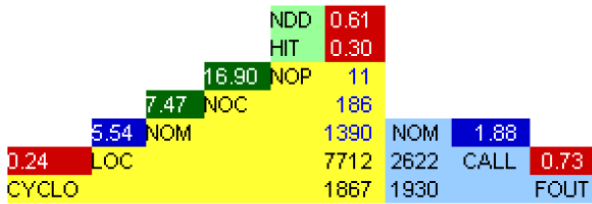


Figure 7

The Overview Pyramid for JHotDraw (Fig.7) reveals a system with significantly smaller dimensions when compared to Apache Ant (Fig. 1), allowing it to serve as a benchmark in this analysis.

In terms of size, JHotDraw contains approximately 186 classes (NOC), 1390 methods (NOM), and 7712 lines of code (LOC), organized into 11 packages (NOP). These values confirm that JHotDraw is a compact system, making it easier to analyze and reason about.

However, despite its smaller size, the system exhibits higher complexity levels. The CYCLO/LOC ratio (0.24) is slightly higher than that of Apache Ant (Fig.1), indicating more complex control flow per line of code. Additionally, the LOC/NOM ratio (5.54) suggests shorter methods, but not necessarily simpler ones.

A particularly notable aspect is the NOM/NOC ratio (7.47), which is higher than in Apache Ant (Fig. 1). This suggests that classes tend to concentrate more methods, potentially indicating heavier responsibilities per class and possible risks of reduced cohesion.

Regarding inheritance, JHotDraw (Fig. 7) shows more pronounced use of hierarchical structures, with AHH (0.30) and ANDC (0.61) both higher than in Apache Ant (Fig. 1). This indicates deeper and wider inheritance hierarchies, which may improve reuse and extensibility but can also increase system complexity [1] and make understanding class relationships more difficult.

In terms of coupling, the CALL/NOM ratio (1.88) reveals a relatively high level of method interaction. Moreover, the FOUT/LOC ratio (0.73) indicates high coupling dispersion, similar to Apache Ant. This suggests that method calls frequently involve a wide range of external classes, increasing interdependencies within the system.

In summary, JHotDraw (Fig. 7) is a smaller but structurally denser system, characterized by higher method concentration per class, more extensive use of inheritance, and significant coupling dispersion. These aspects may make the system more flexible but also potentially harder to maintain and evolve.

3.2.1 Classic Metric View

Class: MDI_DrawApplication				
Me...	Metrics...	Description	Value	Regular...
○ CLOC	Statistic	Lines of Code	207	
○ CCC	G. Ann C...	Cognitive Complexity	6	[0..32]
○ CHVL	Halstead...	Halstead Volume	1259,5479	
○ CHD	Halstead...	Halstead Difficulty	72,6923	
○ CHL	Halstead...	Halstead Length	196	
○ CHEF	Halstead...	Halstead Effort	91559,44...	
○ CHVC	Halstead...	Halstead Vocabulary	86	
○ CHER	Halstead...	Halstead Errors	0,6771	
○ WMC	Chidamb...	Weighted Methods ...	28	[0..12]
○ DIT	Chidamb...	Depth Of Inherita...	7	[0..3]
○ CBO	Chidamb...	Coupling Between ...	16	[0..14]
○ RFC	Chidamb...	Response For A Class	55	[0..45]
○ LCOM	Chidamb...	Lack Of Cohesion O...	2	
○ NOC	Chidamb...	Number Of Children	1	[0..2]
○ NOA	Lorenz-K...	Number Of Attributes	242	[0..4]
○ NOO	Lorenz-K...	Number Of Operati...	834	
○ NOOM	Lorenz-K...	Number Of Overrid...	7	[0..3]
○ NOAM	Lorenz-K...	Number Of Added ...	14	
○ SIZE2	Li-Henry ...	Number Of Attribut...	954	
○ NOM	Li-Henry ...	Number Of Methods	23	[0..7]
○ MPC	Li-Henry ...	Message Passing C...	59	
○ DAC	Li-Henry ...	Data Abstraction Co...	4	
○ ATFD	Lanza-M...	Access To Foreign ...	7	[0..6]
○ NOPA	Lanza-M...	Number Of Public A...	0	[0..3]
○ NOAC	Lanza-M...	Number Of Accesso...	2	[0..4]
○ WOC	Lanza-M...	Weight Of A Class	0,381	[0.5..1.0]
○ TCC	Bieman-...	Tight Class Cohesion	0,0381	[0.33..1.0]
○ NCSS	Chr. Cle...	Non-Commenting S...	84	[0..1000]
○ CMI	Maintain...	Maintainability Index	27,3249	

Figure 8

The Classic Metric View (Fig.8) confirms JHotDraw's structural density despite its smaller size. Notably, WMC reaches 286 and RFC 284 at the class level for the worst offenders, both well above normal thresholds. CBO peaks at 71, indicating strong inter-class dependencies concentrated in specific components. NOA = 242 in MDI_DrawApplication alone suggests extreme attribute accumulation in a single class. These values point directly to God Class candidates and motivate a closer inspection of individual classes.

3.2.2 Metrics Distribution Chart

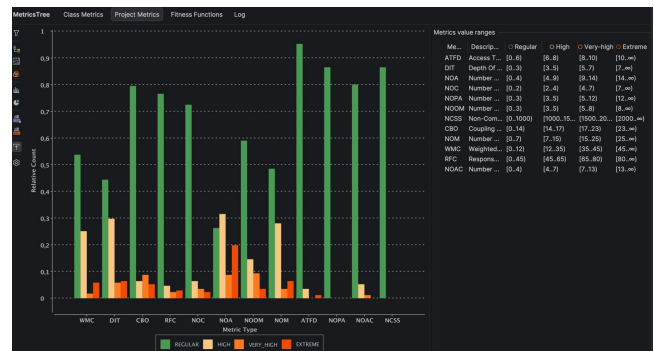


Figure 9

The Metrics Distribution Chart (Fig.9) confirms that most classes fall within the Regular range across all metrics. However, the tail of High, Very-high and Extreme values is significant — particularly for WMC (6.8% Extreme) (Fig. 12), NOM (7.4% Extreme) (Fig. 11) and CBO (~20% above normal combined) (Fig 10). This concentration pattern is typical: a small number of classes drive most of the quality risk, while the majority of the system remains healthy.

3.2.3 CBO Distribution

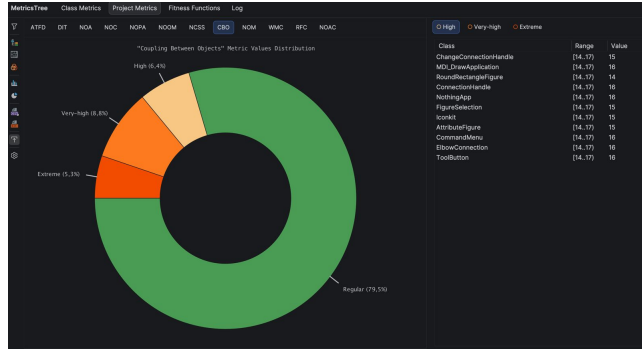


Figure 10

On CBO Distribution (Fig.10), 79.5% of classes have regular coupling. However, the remaining 20.5% — distributed across High (6.4%), Very-high (8.8%) and Extreme (5.3%) — represent meaningful architectural risk. The most coupled classes include MDI_DrawApplication, ConnectionHandle, CommandMenu and ElbowConnection, all with CBO = 16, exceeding the normal ceiling of 14. These classes act as structural hubs whose modification would likely propagate side effects throughout the system.

3.2.4 NOM Distribution

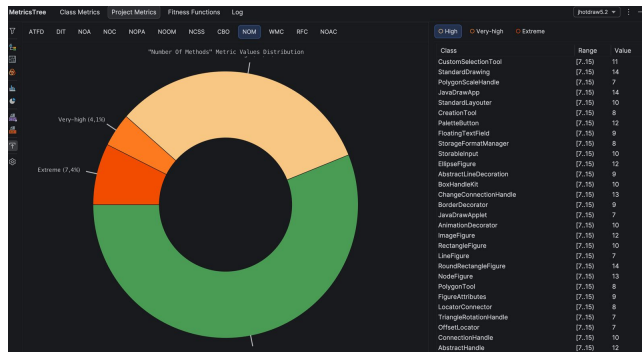


Figure 11

On NOM Distribution (Fig.11), most classes have a regular number of methods, but 7.4% reach Extreme values. Classes such as StandardDrawing (14), RoundRectangleFigure (14) and MDI_DrawApplication (23) stand out. High NOM combined with high WMC — as seen in MDI_DrawApplication — indicates classes that are both large and logically complex, the most problematic combination for maintainability.

3.2.5 WMC Distribution

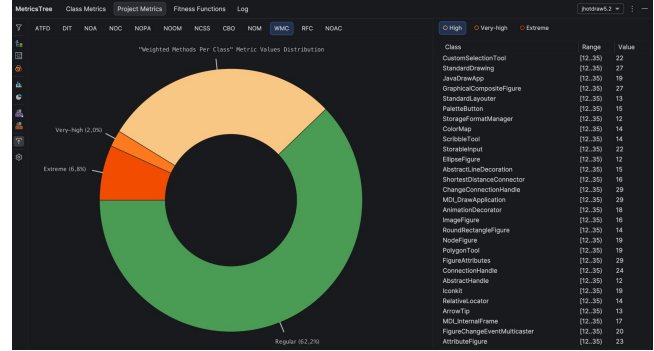


Figure 12

On WMC Distribution (Fig. 12), 62.2% of classes fall within the Regular WMC range. The 6.8% Extreme tail is the most concerning finding. ChangeConnectionHandle (29), MDI_DrawApplication (29) and FigureAttributes (29) represent the complexity hotspots of the system. Their high WMC, combined with elevated CBO and NOM values, strongly suggests God Class or Brain Class disharmonies and makes them the primary candidates for refactoring.

4. CONCLUSION

The analysis of Apache Ant and JHotDraw confirms that both systems present acceptable average metrics, but these values hide a small set of critical components that dominate structural risk. Rather than widespread degradation, the main issues are concentrated in outliers [1], making detailed, class-level analysis essential.

Apache Ant is a large system (1030 classes, 41617 LOC) with controlled average complexity (CYCLO = 0.22) and shallow inheritance. Its main weakness lies in coupling dispersion, reflected in high CBO and WMC outliers. These problematic classes are spread across the system, which makes them less visible and harder to address systematically. As a result, Ant's risks are diffuse, affecting maintainability in a more gradual but pervasive way.

JHotDraw, although smaller (186 classes, 7712 LOC), is structurally denser. It shows deeper inheritance (AHH = 0.30) and higher method concentration (NOM/NOC = 7.47), indicating heavier responsibilities per class. Its main issue is not dispersion but concentration: a few classes, especially MDI_DrawApplication, accumulate extreme values across size, complexity, coupling, and cohesion. These act as clear hotspots, but are more difficult to refactor due to their central role and tight integration.

Overall, both systems follow the same pattern: healthy averages masking a problematic minority. Apache Ant distributes its issues across the codebase, while JHotDraw concentrates them in a few critical components. The combined use of MetricsTree [4] and inFusion [5] was key to identifying this, as high-level views alone would not reveal these structural risks without deeper inspection of metric distributions and outliers.

5. REFERENCES

- [1] Michele Lanza and Radu Marinescu. Object-Oriented Metrics in Practice. Springer, 2006.
- [2] Martin Fowler. Refactoring: Improving the Design of Existing Code. Addison-Wesley, 1999.
- [3] Thomas J. McCabe. A Complexity Measure. IEEE Transactions on Software Engineering, 1976.
- [4] MetricsTree IntelliJ Plugin Documentation. Accessed 2026.
- [5] inFusion Tool Documentation. Accessed 2026.